

Classroom Programming Challenge Platform

Refined Project Planning Document

1. Project Overview

The Classroom Programming Challenge Platform is a classroom-focused web application designed to help professors launch live programming problems to students during class. A professor selects a problem from a shared problem database, launches it to a course, and students submit their solutions through an in-browser coding environment. Submissions are sent directly to the professor as a one-time response and are reviewed anonymously for classroom discussion.

The platform is intended to promote participation, live practice, and constructive discussion while avoiding public embarrassment for incorrect answers. Rather than relying on automated compilation, execution, or test-case grading, the platform emphasizes instructor-guided review of student thinking and solution attempts.

2. Core Purpose

The system is intended to support the following teaching workflow:

A professor teaches a programming topic.

The professor searches for and selects a matching lesson or problem from the platform's database.

The professor launches the problem to a specific course.

The problem is sent to all student accounts associated with that course.

Students complete the problem in a browser-based coding interface.

Students submit their solution one time.

The submission goes directly to the professor for review.

The professor sees a live stream of anonymous student responses.

Incorrect or incomplete solutions can be reviewed in front of the class without revealing student identity.

The platform is designed to encourage participation and learning through guided review, not punishment. Incorrect answers are not meant to shame students and should never display student identity during classroom review.

3. Primary Goals

Instructional Goals

- Allow professors to quickly deploy live coding problems during class
- Encourage active student participation
- Enable anonymous review of incorrect solutions
- Support attendance tracking separately from grading
- Promote learning without penalizing practice mistakes
- Shift emphasis from automated grading to instructor-led discussion

Technical Goals

- Provide a clean browser-based coding experience
- Support course-based professor/student account relationships
- Deliver one-time student submissions directly to the professor
- Display submission results in real time or near real time
- Be deployable on a scalable cloud platform such as AWS

4. User Roles

Professor

A professor can:

- Sign in using Google
- Create or manage courses
- Associate themselves with one or more courses
- Search and launch problems
- Set how long a problem remains active
- View incoming student submissions anonymously
- Review selected submissions
- View student names separately for attendance and roster tracking
- Submit new problems into a review pipeline for inclusion in the shared database
- Relaunch previously used problems from launch history
- Remove students from a course
- Reset a course roster

Problems submitted by one professor and approved through the system's content process can be made available to all professors.

Student

A student can:

- Register or sign in using Google
- Join a course through a professor-generated QR code or class access link
- View active assigned problems for their course
- Open a live problem
- Write and submit a solution in the browser
- Submit once per launch
- Return to the student dashboard after submission
- View whether a problem is active, submitted, or archived

Administrator

An administrator role has not been fully defined yet.

Possible responsibilities may include:

- Managing global problem content
- Reviewing and approving submitted problems
- Managing professor permissions
- Monitoring system health
- Handling abuse or moderation
- Managing billing or paywall features

Admin role and scope: {TO BE DECIDED}

5. High-Level System Architecture

Proposed Stack

- Frontend rendering: Jinja2 templating
- Backend framework: Flask
- Database: PostgreSQL
- Reverse proxy: Nginx
- Hosting: AWS
- Authentication: Google OAuth
- Browser code editor: embedded editor component such as Monaco or CodeMirror {TO BE DECIDED}
- Live update mechanism: polling, Server-Sent Events, or WebSockets {TO BE DECIDED}

Architectural Direction

The platform will not compile, execute, or automatically grade student code in the MVP. Instead, it will function as a structured one-time submission and live classroom review system. This

removes the need for a code execution engine in the first version and significantly reduces infrastructure and security complexity.

Deployment Considerations

The platform would likely be hosted behind Nginx as a reverse proxy, with Flask serving the application and PostgreSQL storing users, courses, problems, launches, submissions, and attendance data.

AWS is a reasonable hosting target, but the exact services are not yet finalized.

Possible AWS services may include:

- EC2 for app hosting
- RDS for PostgreSQL
- S3 for static assets, exports, or backups
- CloudFront: {TO BE DECIDED}
- Route 53: {TO BE DECIDED}
- ECS / Docker deployment: {TO BE DECIDED}

6. Functional Requirements

6.1 Authentication and Account Management

Professor Authentication

- Professors log in using Google
- Professors register and sign in through Google OAuth
- Professors can be associated with one or more course IDs

Student Authentication

- Students register and sign in using Google
- Students join a course through a QR-code-based or access-link-based enrollment flow
- Once registered and linked to a course, future logins take them directly to the student view

Defined Decisions

- Non-Google login support: no
- Password fallback login: no
- Account recovery flow: professor generates reset link
- Multi-factor authentication: no
- Students can belong to multiple courses: yes
- Professors can teach multiple sections of the same course: yes

6.2 Course and Enrollment Management

Course Relationships

- A professor account is tied to one or more course IDs
- Student accounts are tied to one or more course enrollments
- Launching a problem sends it to all students associated with that course

Enrollment Flow

- Professor generates a QR code or access link
- Student scans or opens the link
- Student creates or links their account using Google
- Student is associated with that course

Defined Decisions

- Professors can manually add or remove students: yes
- Professors can reset a course roster: yes
- There is a course roster management screen: yes
- Roster screen shows student names and whether they completed the day's assigned problems
- Daily completion example: 2/3 if a student submitted 2 out of 3 assigned during the current calendar day
- Course join codes expire: yes, after 1 day
- QR codes are reusable: yes
- Courses persist across semesters: no
- Students can switch courses: only if removed by one professor and enrolled by another

6.3 Problem Database

Problem Types

The system supports searchable programming problems such as:

- Write the code snippet given a prompt
- Find the bug in a code snippet
- Finish the code given a snippet

Problem Metadata

Each problem should include:

- Title
- Keywords or tags
- Short description

- Brief description of possible solution methods
- One vague hint for struggling students
- Difficulty level: easy, medium, hard
- Topic category such as data structures, bugs, algorithms, etc.
- Programming language designation
- Starter code when applicable
- Expected runtime or constraints: {TO BE DECIDED}
- Sample input/output: {TO BE DECIDED}

Search

Professors should be able to search by:

- Title
- Keywords
- Topic
- Difficulty
- Language

Defined Decisions

- Initial supported language in problem library: Python
- Future language support: yes, including languages like C
- Problems are manually authored per language and are not automatically converted
- Hidden test cases are no longer part of the primary workflow
- Professors can edit/customize existing problems for the current session only
- Session edits are not saved back to the shared database
- Professors can create course-specific problems through the same submission process
- No code content is private if submitted into the centralized shared database
- Problem ownership and approval workflow: centralized database of all submitted and approved problems

6.4 Launching a Problem

Launch Flow

From the home view, the professor can:

- Select a problem
- Press launch
- Send the problem to all students in the selected course
- Configure how long the problem remains active, up to 2 hours

Launch Configuration

Potential launch settings include:

- Active duration
- Course or section target
- Whether late submissions are allowed: no, only while active
- Whether students can resubmit: no
- Hint release timing: 5-minute delayed hint
- Whether students receive post-submission correctness feedback: no automated correctness in MVP
- Whether a problem can be scheduled in advance: yes, during the current day only

Defined Decisions

- Minimum allowed launch time: 1 minute
- Professor can end a problem early: yes, from the submission stream view using a “complete” control
- Multiple problems can be active at once per course: yes
- Launches can be duplicated or relaunched later: yes
- Launch history page exists with relaunch button: yes

6.5 Student Work and Submission Experience

Student View

Students log in and see a list or stream of all currently active problems assigned to them.

For each problem, the student can:

- Open the problem
- View the problem title
- View the brief description
- View the hint when available
- Use an in-browser development area to write code
- Submit their solution one time
- Return to the student dashboard after submission

Problem View Fields

- Auto-filled student name field
- Browser-based development field or code editor
- Problem title
- Problem description
- Optional starter code
- Hint area
- Submission button

Defined Decisions

- Students see their own correctness result after submitting: no automated correctness in MVP
- Students can edit and resubmit after first submission: no
- Autosave exists: no
- Syntax highlighting in coding field: yes
- stdin/stdout testing support: no for MVP
- Student chooses programming language: no
- Python only for now
- Future professor-selected languages: yes, once implemented
- Name field is auto-filled from account data
- Attendance is marked only on submission

6.6 Submission Evaluation

Updated Core Requirement

Each student submission is sent directly to the professor as a one-time response. The platform does not compile code, execute code, or run hidden test cases in the MVP.

Evaluation Model

Submissions are reviewed by the professor rather than automatically scored through a technical execution engine.

Possible ways the professor may interpret submissions:

- Correct
- Partially correct
- Incorrect
- Incomplete
- Good discussion example
- Needs follow-up

Defined Decisions

- Automated compilation: no
- Automated execution: no
- Automated test-case scoring: no
- AI-assisted correctness analysis: {TO BE DECIDED}
- Manual professor review: yes
- One-time submission model: yes
- Compile/runtime error handling: not applicable as a platform-level grading feature in MVP
- Students may still type syntactically invalid code and submit it
- Plagiarism/similarity detection: no

This change intentionally simplifies the platform into a live instructional submission system rather than an online judge.

6.7 Professor Submission Stream

Professor View

The professor sees a stream of incoming submissions with:

- No student names shown
- Submission timestamp
- Problem title or launch reference
- Anonymous submission card
- Manual review status or instructor label {TO BE DECIDED}
- Lowest-quality or most recent submissions surfaced based on sorting mode {TO BE DECIDED}

Because there is no automated grading in the new design, the professor stream should focus on organizing and reviewing incoming answers rather than displaying computed correctness percentages.

Defined Decisions

- Submissions update live without refresh: yes
- Stream includes submission timestamps: yes
- Stream can be filtered: yes
- Filter dimensions may include submission status, reviewed/unreviewed, or professor-assigned category {TO BE DECIDED}
- Stream shows compile/runtime error type: no longer applicable in MVP
- Stream shows language used: no
- Professor can pin a submission for later discussion: yes
- Professor can reveal student identity privately for follow-up: no

6.8 Review View

Review Flow

When a professor selects a submission from the professor stream, the review view displays the submitted answer for class discussion.

Core Principle

The review view must preserve anonymity. Student names must not be visible during classroom review.

Defined Decisions

- Review view shows the original prompt alongside the answer: yes
- Review view highlights failing sections or bugs automatically: no
- This should remain a teaching moment led by the professor
- Professor can project multiple submissions side by side: up to 2
- Professor annotation tools: no need in MVP
- Reviewed answers can be saved for later instruction: yes
- Scoring breakdown shown: no automated scoring in MVP
- Next/previous submission controls: yes, likely arrows in the top-left

6.9 Attendance / Roll Call

Attendance Requirement

A separate section logs student names for attendance and participation tracking.

Important Distinction

- Student names are used for attendance
- Student names are not shown alongside answers in professor or review views

Defined Decisions

- Attendance is based on submission only: yes
- Attendance cannot be marked manually by professor: no manual marking
- Attendance can be exported: yes
- Attendance credit is proportionate to problems submitted versus assigned
- Attendance ties to the current class day's assigned problems
- Launch-level or class-session attendance record structure: {TO BE DECIDED}

7. UI / View Breakdown

7.1 Login View

Purpose

Entry point for professors and students.

Features

- Google login
- Google registration
- Potential future paywall integration
- Public landing page with platform description and core features
- Buttons for contact, create account, and login

Defined Decisions

- Only professor accounts can be created directly through signup
- Student account creation is locked behind class access link or QR join flow
- Existing student accounts can link new classes through professor-provided access

7.2 Home View

Purpose

Main dashboard for professors.

Features

- Searchable list of problems
- Problem cards with title, keywords, and description
- Launch button
- Professor dashboard shortcut in the top-right
- Active time configuration up to 2 hours
- Favorites support
- Saved search presets loaded from the database

Defined Decisions

- Pagination: yes
- 20 problems per page
- Favorites appear at the top of the list
- Topic filters and saved presets: yes
- Hint availability indicator: {TO BE DECIDED}

7.3 Setup View

Purpose

Allows professors to onboard students into a course.

Features

- Generate QR code or access link
- Link students to course through signup flow
- View current roster
- Remove students from course
- Reset roster

Defined Decisions

- Setup page shows current roster: yes
- Students can be removed here: yes
- Setup availability throughout semester: configurable by professor

7.4 Professor View

Purpose

Live classroom instructor dashboard.

Features

- Stream of anonymous submissions
- Live incoming updates
- Review button for each submission
- Separate attendance list with student names
- Completion control to end a problem early

Defined Decisions

- Projector-friendly mode: default
- Refresh/live sync status: live
- Pause incoming display: no need

7.5 Review View

Purpose

Detailed instructor-only or classroom display of a chosen answer.

Features

- Displays selected submission
- Preserves anonymity
- Shows original prompt with answer
- Supports side-by-side comparison for up to 2 submissions

Defined Decisions

- Scoring breakdown: no automated scoring in MVP
- Next/previous submission controls: yes
- “Good discussion example” marking: no need in MVP

7.6 Student View

Purpose

Main student dashboard.

Features

- List of currently active assigned problems
- Ability to begin solving a problem
- Status labels such as active, submitted, archived

Defined Decisions

- Completed problems remain visible: yes
- Completed and expired problems move to archive: yes

7.7 Problem View

Purpose

Student coding workspace.

Features

- Auto-filled name field
- Problem name
- Brief description
- Optional starter code
- Hint display
- Code editor area
- Submit button
- Return to student dashboard after submission

Layout Decision

- Left half: title, description, optional hint
- Right half: name field at top and code editor below

Defined Decisions

- Layout is split-pane
- Timer shown: no
- Run/test button before final submit: no
- Hint button visibility: visible by default

8. Data Model Overview

A likely starting database design would include:

Users

- user_id
- google_oauth_id
- name
- email
- role (professor/student/admin)
- created_at

Courses

- course_id
- course_name
- professor_id
- term_or_semester
- created_at

Course_Enrollments

- enrollment_id
- course_id
- user_id
- joined_at

Problems

- problem_id
- title
- description
- keywords
- hint
- problem_type
- language
- difficulty
- starter_code
- created_by
- approval_status
- created_at

Problem_Launches

- launch_id
- problem_id
- course_id

- launched_by_professor_id
- start_time
- end_time
- status
- settings_json

Submissions

- submission_id
- launch_id
- student_id
- student_display_name_for_attendance
- submitted_code
- submitted_at
- professor_review_label {TO BE DECIDED}
- professor_notes {TO BE DECIDED}
- pinned_flag {TO BE DECIDED}
- archived_for_instruction_flag {TO BE DECIDED}

Attendance

- attendance_id
- launch_id_or_class_session_id {TO BE DECIDED}
- student_id
- attendance_name
- recorded_at

Problem_Submissions_For_Approval

- approval_submission_id
- problem_id_or_draft_id
- submitted_by_professor_id
- review_status
- reviewer_id {TO BE DECIDED}
- submitted_at
- reviewed_at {TO BE DECIDED}

Because the system no longer depends on automated execution or hidden test cases in the MVP, a **Problem_Test_Cases** table is optional and may be deferred unless future grading features are added.

9. Security and Privacy Considerations

Because student submissions are tied to identities internally but shown anonymously in professor review contexts, privacy must be clearly enforced.

Required Privacy Behaviors

- Student identity must not appear in professor submission streams
- Student identity must not appear in class review view
- Attendance data must be stored separately from review-facing data
- Only authorized professors should be able to launch and view course-specific submissions
- Student-facing and professor-facing access must be role-checked

Technical Security Concerns

- Secure Google OAuth handling
- Session management
- CSRF protection
- Access control between courses
- Protection of student-submitted content
- Full audit logging
- Rate limiting: {TO BE DECIDED}
- Content moderation for inappropriate submissions: {TO BE DECIDED}

Defined Decisions

- Submission retention period: 1 month
- Professors can download submissions: yes, export as `.py`
- FERPA-related requirements: {TO BE DECIDED}
- Student data deletion/export: no in initial version

Even if grades are not stored, student identities, participation records, and course-linked submissions still create privacy obligations, so institutional policy review is still advisable.

10. Browser Development Field

This remains an important design area, but it is now much simpler than a full code execution platform.

Needed Capability

Students need an in-browser field where they can write code and submit it directly to the professor.

Required Features

- Syntax highlighting
- Language-aware editing for Python
- Support for starter code
- Clean full-classroom usability
- Reliable one-time submission flow

Key Remaining Decisions

- Which editor component to use: {TO BE DECIDED}
- Whether future versions will support multiple languages: yes
- Whether future versions will add run/test functionality: {TO BE DECIDED}
- Whether the editor should include linting or formatting help: {TO BE DECIDED}
- Whether submission should warn before final one-time submission: {TO BE DECIDED}

Because code is not compiled or executed in the MVP, the browser development field is primarily an input and submission interface rather than an online IDE.

11. Future Features

These are not fully defined but are natural later extensions:

- Paywall / subscription model
- Analytics dashboard for professors
- Saved class performance trends
- Problem authoring tools
- Custom course problem banks
- Student progress tracking
- Export attendance and submission reports
- LMS integration such as Canvas, Blackboard, or Moodle: {TO BE DECIDED}

12. Open Questions / Missing Decisions Summary

These are the biggest items still needing definition:

- Live update mechanism — polling, SSE, or WebSockets
- Exact professor-side review labels and workflow
- Whether AI-assisted submission summarization or categorization should exist
- Which editor component will power the coding field
- Whether future versions will support multiple languages beyond Python
- Whether professors can author and approve problems directly or require admin review
- Exact attendance record structure — per launch or grouped by class session
- Hosting architecture — exact AWS services and deployment model
- Privacy/compliance requirements — FERPA or institution-specific student data rules
- Future monetization/paywall design

- Admin tools and moderation features
- Whether the one-time submission flow should have a confirmation screen
- Whether hints should always be shown or be delayed based on launch settings

13. Suggested MVP Scope

To keep the first version realistic, the MVP should include:

- Google login for professors and students
- Course linking through QR code or access link
- Professor problem list and search
- Shared approved problem database
- Problem launch to a course
- Student dashboard of active problems
- Simple browser-based code editor
- One supported language: Python
- One-time student submission
- Anonymous professor submission stream
- Review view for selected answers
- Separate attendance list
- Roster management
- Launch history and relaunch support

The following should be deferred until later:

- Multi-language support
- Automated grading
- Compilation/execution sandboxing
- AI grading
- Advanced analytics
- Paywall
- Admin dashboard expansion
- LMS integrations
- Advanced classroom controls

14. Refined Product Summary

This application is a live classroom coding practice platform where professors can launch programming questions to a course, students can submit one-time solutions through a browser-based coding environment, and professors can review anonymous answers with the class to promote discussion and learning. The system separates attendance tracking from answer review so students are encouraged to participate without fear of being singled out for incorrect solutions.

The proposed stack of Flask, Jinja2, PostgreSQL, Nginx, and AWS is a strong fit for this concept. With the updated design, the platform is no longer an online code execution system in its first version. Instead, it is a structured instructional submission and review platform, which makes the MVP much more realistic to build while still preserving the core classroom value.

If you want, I can also turn this into a cleaner **formal software requirements document layout** with sections like **Scope, Functional Requirements, Non-Functional Requirements, Data Model, and MVP vs Future Work** so it looks more like something you would submit for a class or project proposal.